BNamericas Technical Challenge

Sep 30, 2026 · @Juan Diego

How the challenge works

The challenge has two parts on one codebase: you build a small slice of a tool at home, then we extend it together in the interview. It replaces the "around two hours" challenge mentioned in the job description.

Part	Where	Time	What happens
1. Build	At home, in your own time	About an hour of work, likely less with good tooling; returned within a week	You build on our starter repo, in your own GitHub repository
2. Extend	The technical interview, on your machine	About 75 minutes	You walk us through your decisions, then we change the requirements and you adapt the code live

Use AI throughout both parts. It is expected, not tolerated.

Part 1: build it at home

You build a small project editor in front of an API you do not own, in about an hour, and send it back within a week. The full brief is in the starter repo's README; this is the summary.

1. Open the starter repo, github.com/juandiegoantezana-zaga/cms2-hiring-starter, and click **Use this template** to create your own repository. If you make it private, invite `juandiegoantezana-zaga`.
2. Run the upstream records API on your machine, following `upstream/README.md`. It holds about 30 fictional projects, and it has some awkward habits that are part of the task.
3. Build the three pieces below, then send us what the last table lists.

Piece	What it needs to do
Backend (Python, FastAPI)	Sits between the browser and the upstream API: list projects, show one, save edits
Frontend (React + MUI)	A project list, and an edit form for the core fields and the key dates
Safe concurrent editing	Two people may edit the same project at once. Changes to different fields must both survive; on the same field, neither silently wins and the person saving second decides. The upstream gives you no help. Decide what "the same field" means for the key dates, and say why

One hard rule: the browser never calls the upstream API directly.

Where this will run. In production your backend runs as three instances behind a load balancer, and a nightly import job writes to the upstream directly, without going through your backend. You do not need to build either. Your design does need to survive both.

You will extend this code live in the interview. Leave it in the state you would want to work in.

The task is deliberately larger than the time. What you cut, and why, is part of what we read.

Send back	Notes
Your repo link	Everything committed, with your own README: what runs, what does not, how to start it
`DECISIONS.md`	One page: what you chose, what you cut and why, what would break first with 50 editors
Your AI trail	Whatever your tooling left behind, and a short note: how you approached the task before writing code, where the tool carried the work, where you overrode it, what you checked by hand

Part 2: extend it in the interview

The technical interview runs on your code, on your machine, with you sharing your screen. It is with the Head of Engineering and takes about 75 minutes.

Segment	Time	What happens
Walkthrough	15 min	You show what you built, the decisions behind it, and how you worked with AI, on this task and in general
Live change	45 min	We change the requirements, and you adapt the code as you would at work
Your questions	10–15 min	Anything about the team, the system or the role

Come with the environment you actually work in, configured the way you like it, including anything you have added to your AI tooling over time. Have the project running before we start. Nothing is off limits during the session: we want to see how you really work, not how you perform under restrictions.

A second interview, on culture and ways of working, follows with the Head of HR.

What we look for

We read your reasoning more closely than your code. Something small that works and is honestly described beats something large that is oversold.

We look at	What good looks like
Decisions	You name the tradeoff, pick a side, and know what it costs
Failure	The awkward cases are in your design from the start, not patched on later
How you direct AI	You set your tools up for the job, give them real context, check what they produce, and know where you refused to delegate
Your toolchain	You move comfortably between your editor, your AI tooling, git and GitHub, and you pick the right one for each step
Adaptability	When requirements change, the change lands where it belongs, and you revisit earlier decisions when it justifies that
Communication	A colleague could make a decision from your `DECISIONS.md` without asking a follow-up

We do not score UI polish, test coverage percentage, formatting, whether it ran first time, or how much you produced.

Practicalities

- **Questions:** email juan.antezana@bnamericas.com. Asking a good question is fine; we would rather you did than guessed.
- **Your repository:** public or private is your choice. If private, invite `juandiegoantezana-zaga` before you send the link.
- **The data:** every project, company and place in the starter repo is fictional. Nothing in it is confidential.
- **What you need:** a GitHub account, Python with uv, Node, and the AI tooling you normally work with.